Your idea is correct, but let me rewrite it clearly in **technical English** so it reflects a proper engineering task.

Refined Task Description

Target Objective

1. **PDF Processing**

Extract content from PDF files including:

- Text
- Tables
- Structured sections

3. **Data Extraction / Scraping**
4. Parse the extracted PDF data.
5. Clean and normalize the content.
6. **Chunking**
7. Split the extracted content into **semantic chunks** for better retrieval.
8. **Vectorization & Storage**
9. Convert chunks into embeddings.
10. Store them in **Weaviate** with proper indexing.
11. **Indexing**
12. Create collections (schemas) in Weaviate.

Store metadata such as:

- document_name
- page_number
- chunk_id

DMV CORETECH

- source

14. **Retrieval**

15. Perform semantic search over stored chunks.

Short Professional Version

Goal:

Build a pipeline that extracts text and tables from PDF documents, processes and chunks the content, and stores it in **Weaviate** as vector embeddings for efficient semantic search and retrieval.

Typical Pipeline (Industry Standard)

Code

```
PDF
↓
Text + Table Extraction
↓
Cleaning
↓
Chunking
↓
Embedding Generation
↓
Store in Weaviate
↓
Semantic Search / RAG
```

Suggested Tools

DMV CORETECH

Task	Tools
PDF parsing	PyMuPDF / pdfplumber
Table extraction	Camelot / Tabula
Chunking	LangChain / custom splitter
Embeddings	OpenAI / HuggingFace
Vector DB	Weaviate

My recommendation (important for you)

Since you are already working with:

- **LangChain**
- **Agentic RAG**
- **Weaviate**

The **best learning task** is:

> *Build a **PDF → RAG pipeline using Weaviate***

This will teach you:

- document ingestion
- chunking strategy
- vector indexing
- retrieval
- RAG architecture

This is **exactly what companies build**.

DMV CORETECH

If you want, I can also show you the **complete production architecture for:**

PDF → Chunking → Embeddings → Weaviate → RAG Agent

(which is the same architecture used in many AI products).

DMV
CORETECH